

Lecture review

Stack and Queue

Stack and *queue* are two important Abstract Data Types (ADTs). A **stack** stores items in a **last-in, first-out (LIFO)** manner, whereas a **queue** stores items in a **first-in, first-out (FIFO)** manner.

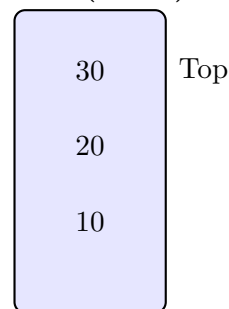
- Stack example: Function calls, undo operations, expression evaluation.
- Queue example: Printer jobs, customer service lines, task scheduling.

The following table summarizes the operations of a stack and a queue.

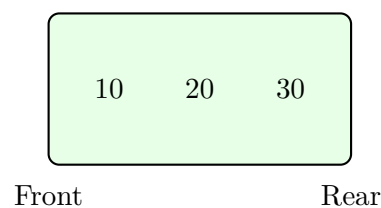
Stack	Queue	Description
push(x)	enqueue(x)	Insert an item x
pop()	dequeue()	Remove an item (top for stack, front for queue)
empty()	empty()	Return true if empty
size()	size()	Return number of stored items

Visual Representation of Stack and Queue

Stack (LIFO)



Queue (FIFO)



We have seen two implementations of stacks and queues in the lecture: **RAStack** and **RAQueue** using *resizable arrays*, and **LLStack** and **LLQueue** using *linked lists*.

From the user's perspective, the implementations are indistinguishable since they provide the same public interface.

Shallow vs Deep Copy: Stack Example

Consider the following class **Stack** that stores data using dynamic memory.

```

class Stack {
public:
    int* data;

    Stack() {
        data = new int[1];
        data[0] = 10;
    }

    ~Stack() {
        delete[] data;
        std::cout << "Memory freed\n";
    }
};

```

The class allocates memory dynamically using a pointer **data**. When such a class is copied, the behavior depends on whether the copy is *shallow* or *deep*.

Shallow Copy

If no copy constructor or assignment operator is defined, C++ performs a **shallow copy** by default.

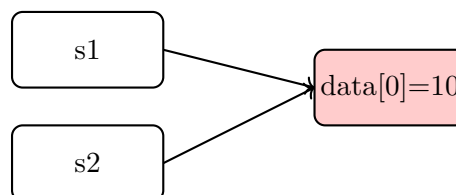
```

int main() {
    Stack s1;
    Stack s2;

    s2 = s1;    // SHALLOW COPY (default)
}

```

Shallow Copy



Deep Copy Using Assignment Operator

To avoid memory sharing, we must allocate new memory and copy values manually.

```

class Stack {
public:
    int* data;

    Stack() {

```

```

    data = new int[1];
    data[0] = 10;
}

// Deep copy assignment operator
Stack& operator=(const Stack& other) {
    if (this != &other) {
        delete[] data;          // free old memory
        data = new int[1];      // allocate new memory
        data[0] = other.data[0]; // copy value
    }
    return *this;
}

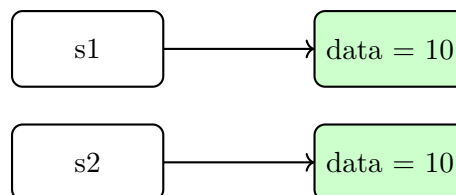
~Stack() {
    delete[] data;
    std::cout << "Memory freed\n";
}
};

int main() {
    Stack s1;
    Stack s2;

    s2 = s1;    // DEEP COPY (safe)
}

```

Deep Copy



Shallow Copy	Deep Copy
Copies pointer only	Copies actual data
Shared memory	Separate memory
Fast but unsafe	Slower but safe
May cause double delete	No memory conflict

Lab exercises

[revise Class and Object concepts](#)

Exercise 1

Power Crystals Arena

You are designing a simple game called *Power Crystals Arena*. Each player collects a pair of magical power crystals that determine their total power.

Game Rule: Each player owns exactly **two crystals**. Each crystal has an integer power level.

Class Design Requirement

Create a class called `CrystalPair` that represents the two crystals of a player.

- The class should contain a dynamically allocated integer array of size 2 to store the power of the crystals.
- Provide a constructor that initializes the two crystal powers.
- Provide a member function `sum()` that returns the total power of the player.

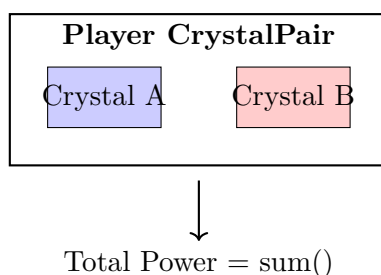
Copying Scenario

During the game, players can use a special ability called *Mirror Spell*, which creates a copy of a player's crystals.

1. In the **practice mode**, a copied player is allowed to share the same crystals (i.e., **shallow copy is acceptable**).
2. In the **battle mode**, modifying one player's crystals must **not** affect the other player. Therefore, a **deep copy is mandatory**.

Tasks:

1. Implement the `CrystalPair` class.
2. Implement a **copy constructor** that performs a **deep copy**.
3. Explain briefly (in comments) why shallow copying would be dangerous in battle mode.
4. Write a test program that: Creates one player, copies it into another player, modifies one player's crystal, shows that the other player's crystals remain unchanged



Exercise 2**Classroom Scenario: Student Alis Merit Position**

During the current semester, a high school conducted **five subject exams**:

1. Mathematics
2. Physics
3. Chemistry
4. Computer Science
5. English

One of the students, **Ali**, is aiming to become the *valedictorian* of the class. Ali wants to know how many students (if any) have obtained a **higher total score** than him across these five subjects.

You are required to design a program using **Object-Oriented Programming** to help determine Alis position in the class.

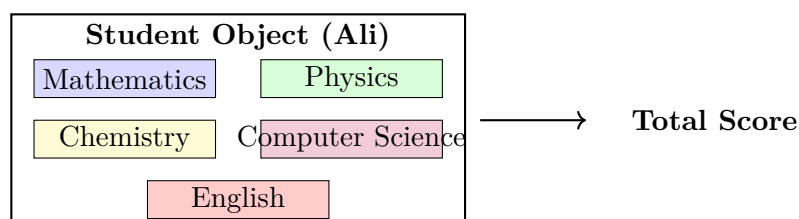
Class Design Requirement:

Create a class named **Student** with the following specifications:

- (a) An instance variable named **Scores** to store a students exam scores for the five subjects.
- (b) A void member function **input()** that reads **five integers** and stores them in **Scores**.
- (c) A member function **calculateTotalScore()** that returns the **sum of all five subject scores**.

Task:

- Create multiple **Student** objects.
- Compute the total score for each student.
- Determine how many students scored higher than **Ali**.

[Access Modifiers](#)**Exercise 3****Secure Bank Account System**

A bank is developing a secure account management system for its customers. For security reasons, sensitive information such as the account number, current balance, and security code must not be accessed directly from outside the system.

To ensure data protection and encapsulation, the bank decides to store all account-related data as private members inside a class.

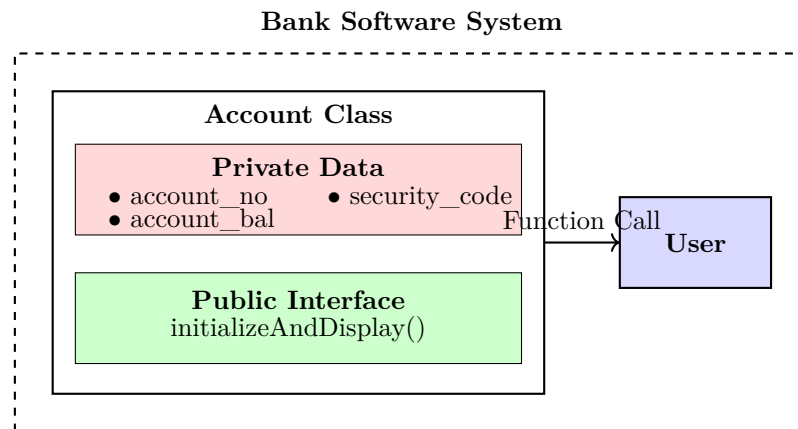
You are required to design a class named **Account** that represents a bank account.

Class Design Requirements:

- The class should contain the following **private** member variables:
 - **account_no** (Account Number)
 - **account_bal** (Current Balance)
 - **security_code** (PIN / Security Code)
- Provide a **public member function** to:
 - Initialize all the account details
 - Display the stored account information in a readable format

Task:

- Create an object of the **Account** class.
- Initialize the account using the public function.
- Display the account details while maintaining proper data encapsulation.



Static and Constant

Exercise 4

Smart Device Factory Object Counter & Safety Rules

You are developing software for a *Smart Device Factory* where multiple devices are produced during a day. Each time a new device object is created, the factory wants to keep track of the **total number of devices manufactured**.

1. Create a class named **Device**.

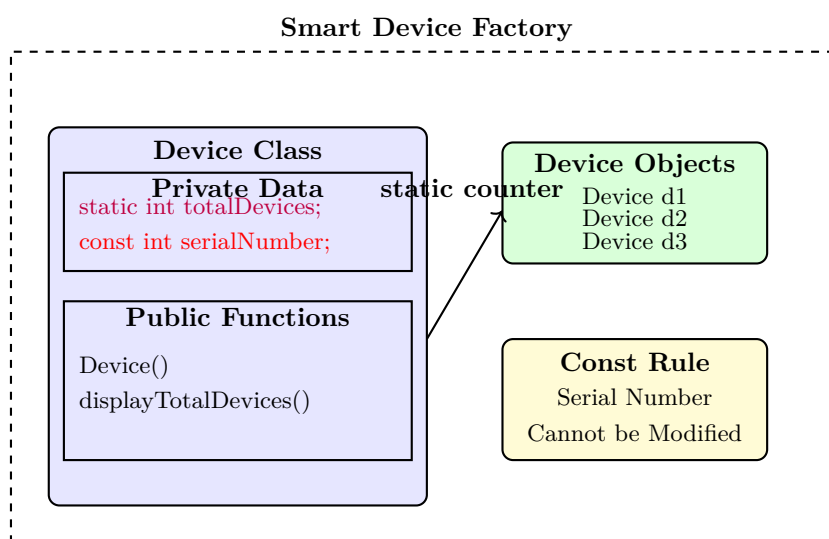
2. Use a **static data member** to count how many **Device** objects have been created.
3. Increment the counter every time a new object is constructed.
4. Provide a public function to display the total number of devices created.

Additionally, every device has a **fixed serial number** that must not be changed once assigned.

1. Use the **const keyword** to declare a serial number for each device.
2. Demonstrate that the serial number cannot be modified after initialization.

Write a complete C++ program to demonstrate both:

- The use of the **static keyword** for object counting
- The use of the **const keyword** to enforce data safety



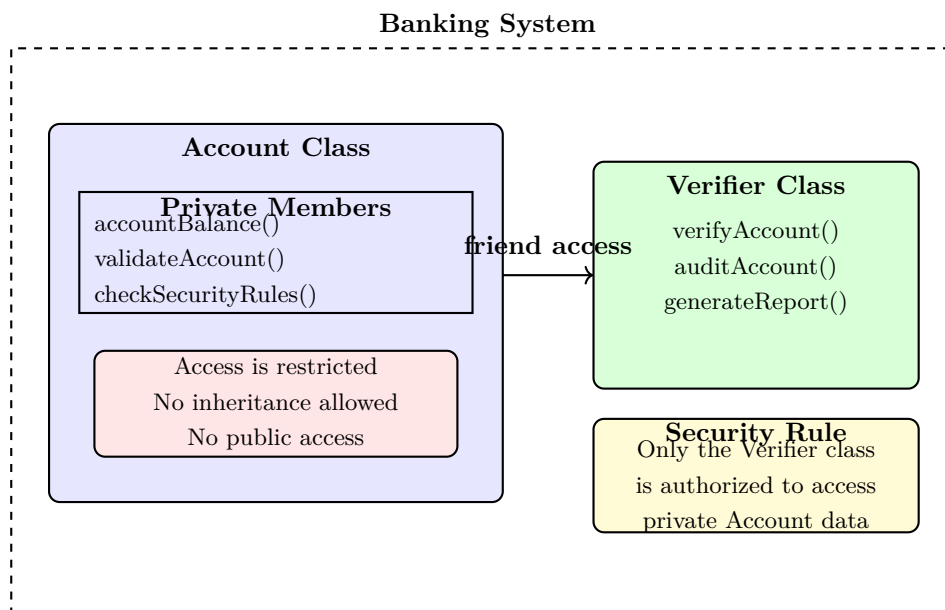
Exercise 5

In a banking system, an **Account** class stores sensitive financial information. For security reasons, this information must remain **private** and must not be accessed through inheritance or public member functions.

Only a specially authorized class named **Verifier** is allowed to access and validate this private information during internal audits.

Task:

- Create a class named **Account** containing a **private function**.
- Create another class named **Verifier** that needs to access this private function.
- Since inheritance is not allowed, implement this access using the **friend class** concept.
- In the `main()` function, create an object of **Verifier** and use it to access the private function of **Account**.

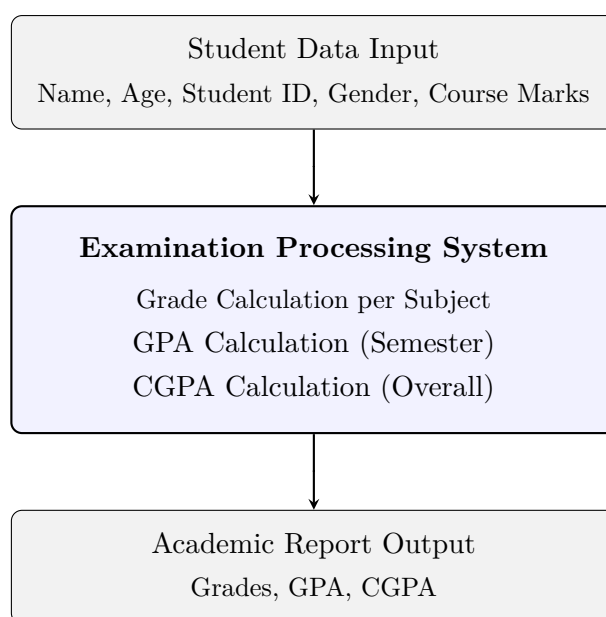
**Exercise 6**

Write a program for a university examination office to record and process student academic data. The program should take the following information as input: Student name, Age , Student ID , Gender, marks obtained in Programming Fundamentals and , marks obtained in the remaining registered courses.

After receiving the input, the program must:

- Display the complete student profile
- Show marks and grade of each subject
- Calculate and display the GPA for the current semester
- Calculate and display the overall CGPA

The GPA and CGPA must be calculated using the formulas provided below.



[Stack and Copy Constructor](#)

Exercise 7

The Bracket Detective

You are a **bracket detective** on a mission! Your goal is to check strings made of parentheses `'()`, square brackets `'[]`, and curly braces `'{'` to see if they are properly balanced.

Write a `**Stack client program**` that:

1. Reads a string of these brackets from standard input.
2. Uses a stack to determine if the brackets are balanced.
3. Prints `true` if they are balanced, `false` otherwise.

What does balanced mean?

A string is considered *balanced* if:

- Every opening bracket has a corresponding closing bracket of the same type.
- Brackets are properly nested.

Examples

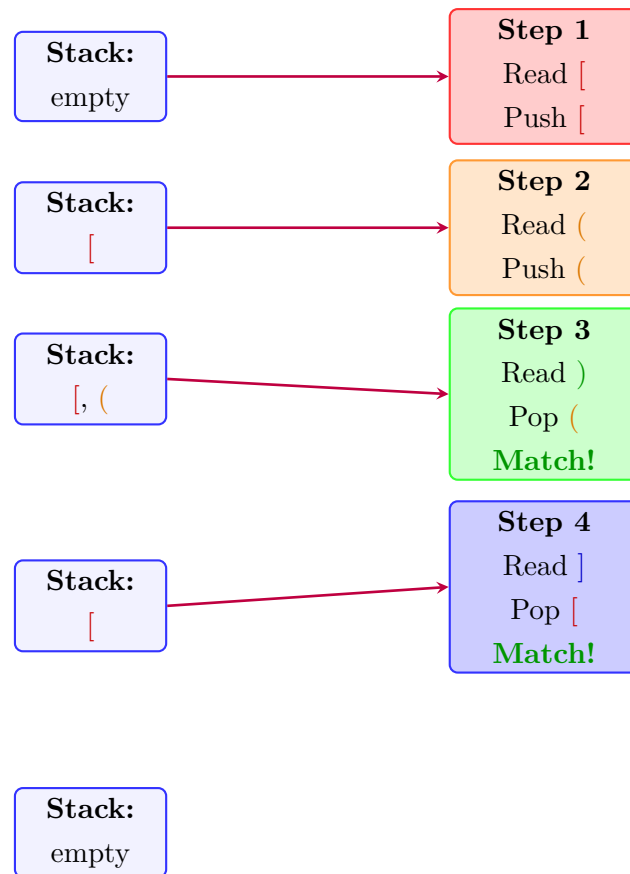
- Input: `[]{}[]{}()` → Output: `true`
- Input: `[]()` → Output: `false`

Hint

Push opening brackets onto the stack. When you see a closing bracket, pop the stack and check if it matches. If the stack is empty at the end and all brackets matched, the string is balanced.

How the Stack Works

Heres a simple diagram showing how a stack checks the string `[]()` step by step:



Explanation of the Diagram

- Step 1: ‘`[`’ is an opening bracket push onto stack.
- Step 2: “`(`” is an opening bracket push onto stack.
- Step 3: “`)`” is a closing bracket pop the stack and check matches “`(`”
- Step 4: “`]`” is a closing bracket pop the stack and check matches ‘`[`’
- Stack is empty at the end string is balanced.

Exercise 8

The Josephus Survival Game

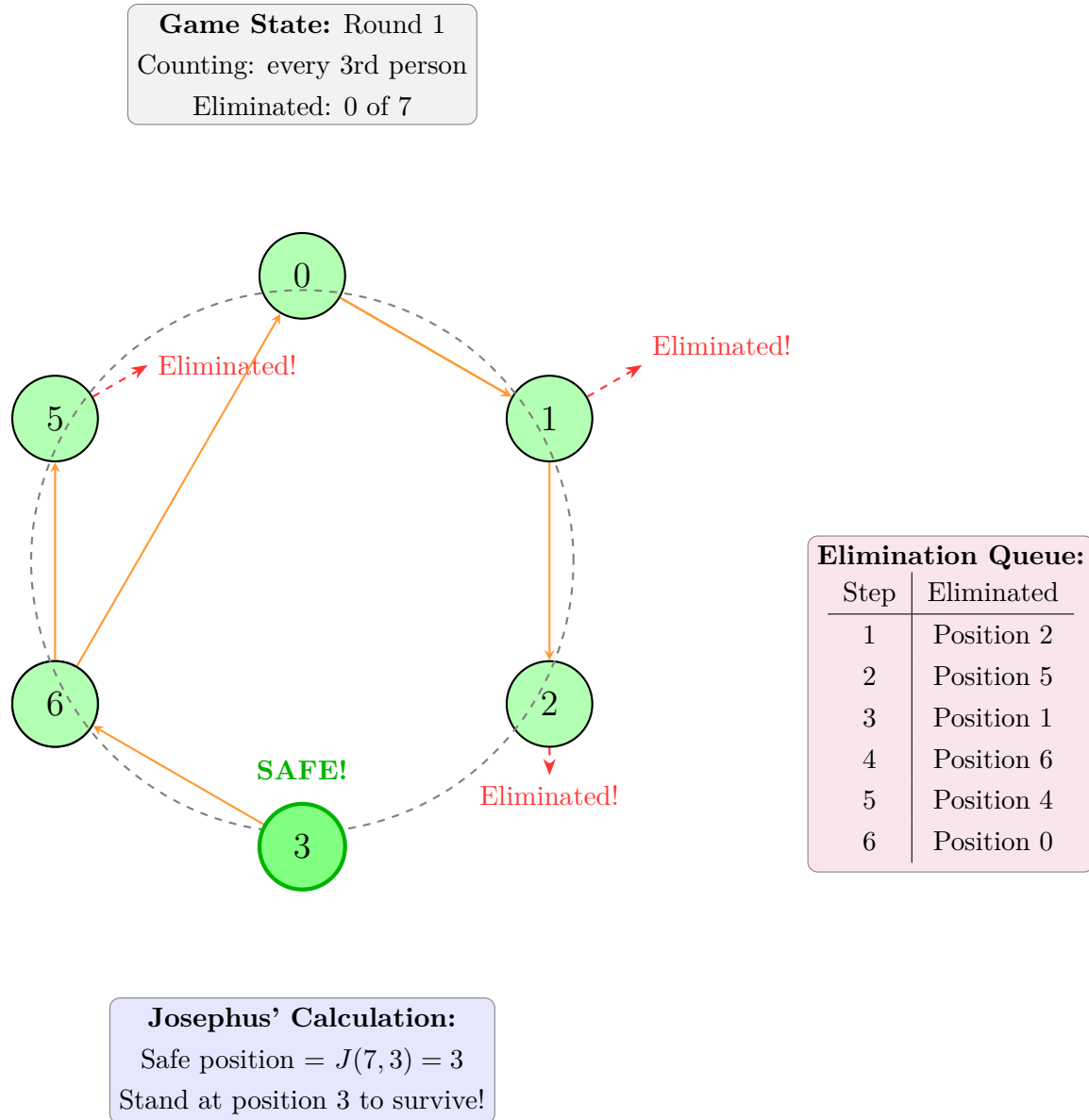
Legend: In ancient Rome, Josephus and his $n - 1$ comrades were trapped in a cave surrounded by Roman soldiers. Rather than surrender, they decided to form a death circle and eliminate every m -th person until only one remained who would then surrender to tell their story. Josephus quickly calculated where to stand to be the survivor!

Game Rules

- **Players:** n people numbered 0 to $n - 1$
- **Elimination rule:** Count m people, eliminate the m -th

- **Circle continues:** Counting resumes with next person after elimination
- **Goal:** Find the **safe position** (where to stand to survive)

Visualization: $n = 7$, $m = 3$ (Example Game)



Interactive Challenge!

- **Warm-up:** $n = 5$, $m = 2$ Safe position?
- **Medium:** $n = 10$, $m = 4$ Where would you stand?
- **Hard:** $n = 41$, $m = 3$ Josephus' actual historical problem!
- **Expert:** Can you find a formula? $J(n, m) = ?$

(Answer: 2)

The Mathematical Trick

Josephus discovered a recurrence relation:

$$J(1, m) = 0$$

$$J(n, m) = (J(n - 1, m) + m) \mod n$$

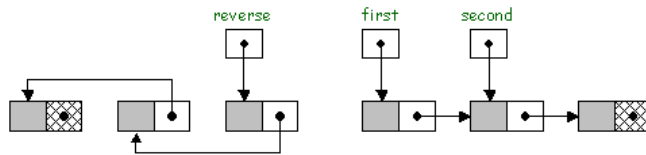
For the quick-witted:

$$J(7, 3) = (J(6, 3) + 3) \mod 7 = \dots = 3$$

Exercise 9

Reverse a linked list. Write a function that takes the first **Node** in a linked list as an argument, and reverses the list, returning the first **Node** in the result.

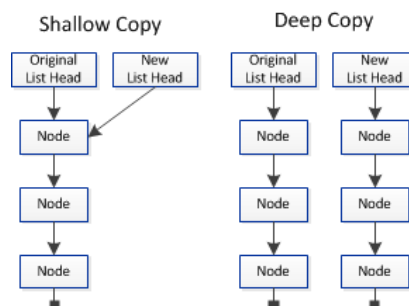
Solution: To accomplish this, we maintain references to three consecutive nodes in the linked list, **reverse**, **first**, and **second**. At each iteration we extract the node **first** from the original linked list and insert it at the beginning of the reversed list. We maintain the invariant that **first** is the first node of what's left of the original list, **second** is the second node of what's left of the original list, and **reverse** is the first node of the resulting reversed list.



Exercise 10

Copy constructor for a stack. Create a new constructor for class **LLStack** (linked-list implementation of stack) so that **LLStack t {s};** makes **t** reference a new and independent copy of the stack **s**. That is, a deep copy of linked list pointed by **s.first** and make **t.first** point to the new list.

With deep copy, you should be able to push and pop from either **s** or **t** without influencing the other.



Exercise 11

Write a recursive function to print a linked list in reverse order. You may not use any loops or iteration.

Exercise 12

A palindrome is a word, phrase, number, or other sequence of characters which reads the same backwards and forwards. Can you determine if a given string, `s`, is a palindrome?

To solve this problem, you must first take each character in `s`, enqueue it in a queue, and also push that same character onto a stack. Once that's done, we must dequeue the first character from the queue and pop the top character off the stack, then compare the two characters to see if they are the same; as long as the characters match, we continue dequeuing, popping, and comparing each character until our containers are empty (a non-match means `s` isn't a palindrome).

Write the following declarations and implementations:

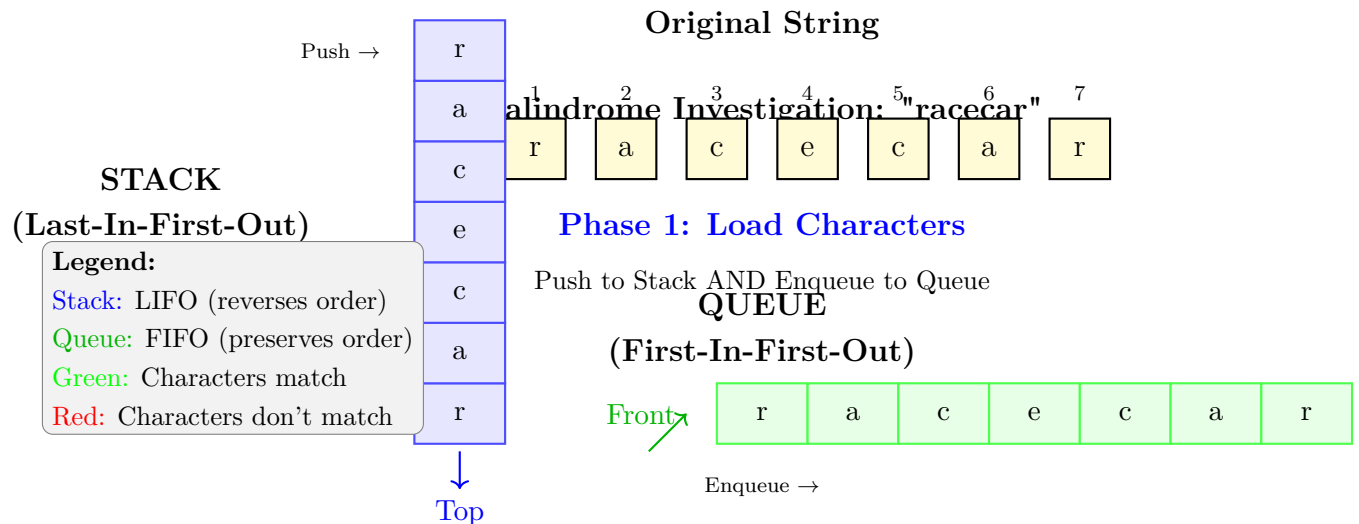
Two instance variables: one for your **stack**, and one for your **queue**.

A void **pushCharacter(char ch)** method that pushes a character onto a stack.

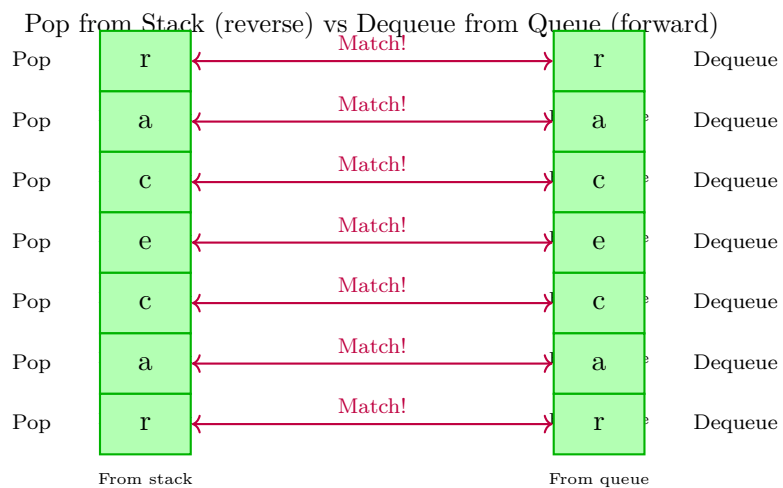
A void **enqueueCharacter(char ch)** method that enqueues a character in the queue instance variable.

A char **popCharacter()** method that pops and returns the character at the top of the stack instance variable.

A char **dequeueCharacter()** method that dequeues and returns the first character in the queue instance variable.

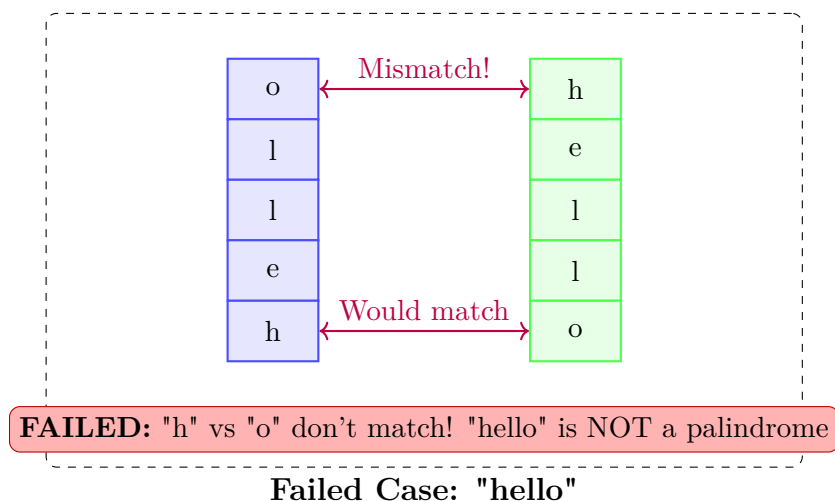


Phase 2: Compare Characters



"racecar" IS A PALINDROME!

All 7 comparisons matched perfectly!

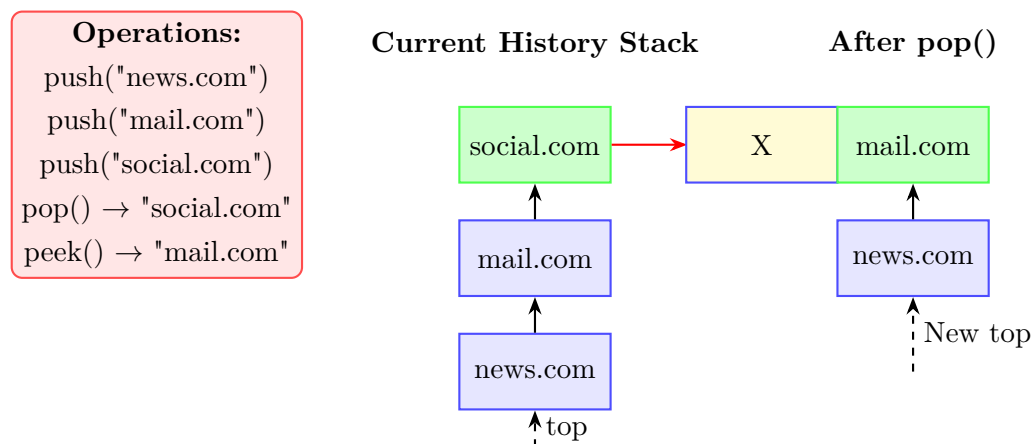


Objective: Implement stack (LIFO) and queue (FIFO) data structures using singly linked lists. Each problem presents a real-world scenario requiring specific operations.

Exercise 13

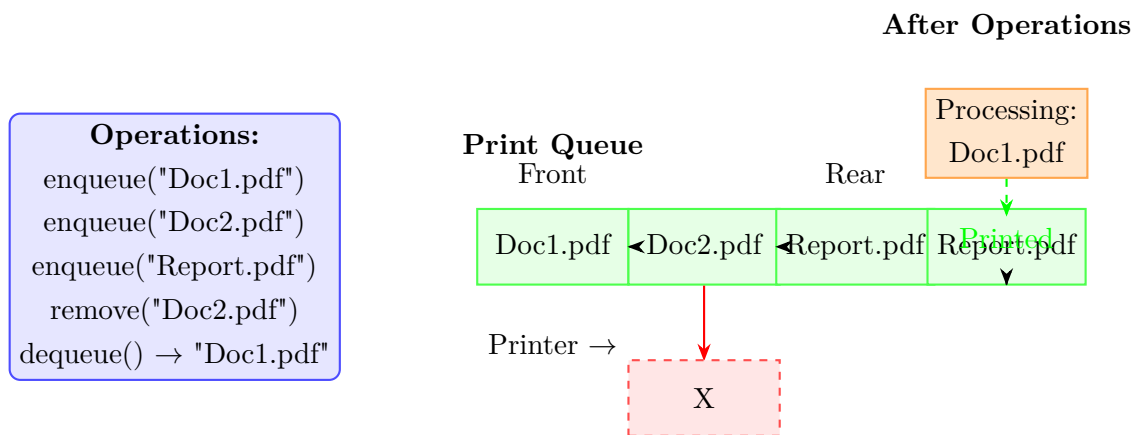
Design a browser history system that allows users to navigate through websites efficiently. The system should support visiting new websites, going back to the previous page, clearing all history, and viewing the current page. Specifically, you are required to implement a **Stack** data structure using a linked list to manage the browser history with the following operations:

- **push(url)**: Adds a newly visited page to the history.
- **pop()**: Goes back to the previous page by removing the current page from the history.
- **peek()**: Retrieves the current page without removing it from the history.
- **isEmpty()**: Checks if the history is empty.
- **clear()**: Clears all browsing history.

**Exercise 14**

Design a shared printer job management system that processes print jobs in the order they arrive (FIFO). The system should support adding new print jobs, processing jobs in arrival order, canceling specific jobs, and viewing the next job without processing it. Specifically, you are required to implement a **Queue** data structure using a linked list to manage the printer jobs with the following operations:

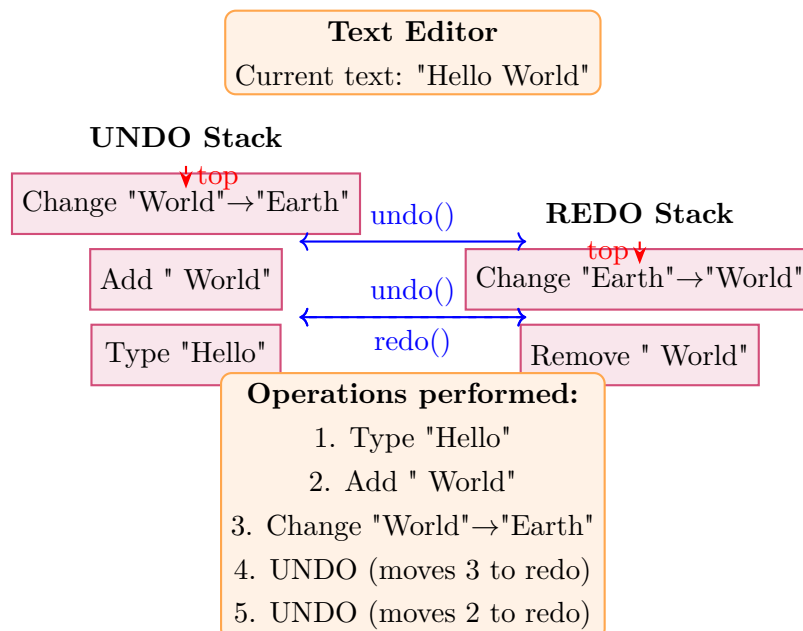
- **enqueue(job)**: Adds a new print job to the queue.
- **dequeue()**: Processes the next job by removing it from the queue.
- **peek()**: Views the next job without removing it from the queue.
- **remove(jobId)**: Cancels a specific job from the queue.
- **isEmpty()**: Checks if the queue is empty.



Exercise 15

Design a text editor system that supports undo and redo functionality using two stacks. Each edit action should be recorded in the undo stack. Performing an undo operation moves the action from the undo stack to the redo stack, while a redo operation moves it back to the undo stack. Any new edit should clear the redo stack. Specifically, implement two stacks (undo and redo) using a linked list with the following operations:

- **doAction(action):** Adds a new edit action to the undo stack and clears the redo stack.
- **undo():** Moves the top action from the undo stack to the redo stack.
- **redo():** Moves the top action from the redo stack back to the undo stack.
- **canUndo()/canRedo():** Checks if the undo or redo stacks are not empty.
- **clearAll():** Clears both the undo and redo stacks.

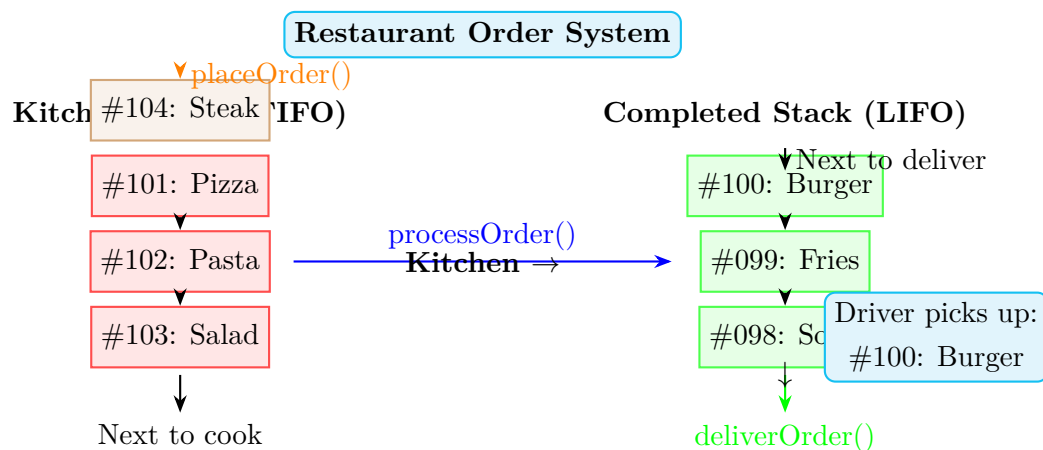


Exercise 16

Design a restaurant order management system that handles both incoming and completed orders. New orders are added to a kitchen queue and processed in the order they arrive (FIFO). Once processed,

orders are moved to a completed orders stack for delivery, where delivery picks up the last completed order first (LIFO). Specifically, implement a combined system using a linked list for both queue and stack with the following operations:

- `placeOrder(order)`: Adds a new order to the kitchen queue.
- `processOrder()`: Moves the next order from the kitchen queue to the completed orders stack.
- `deliverOrder()`: Removes the top order from the completed orders stack for delivery.
- `getNextKitchenOrder()`: Views the next order in the kitchen queue without removing it.
- `getNextDelivery()`: Views the top order in the completed stack without removing it.



Implementation Guidelines

1. For Stack (Problems 13 & 14):

- Maintain **top** pointer (head of list)
- `push()`: Insert at head ($O(1)$)
- `pop()`: Remove from head ($O(1)$)
- Need to handle empty stack case

2. For Queue (Problems 15 & 16):

- Maintain **front** and **rear** pointers
- `enqueue()`: Insert at rear ($O(1)$ with tail pointer)
- `dequeue()`: Remove from front ($O(1)$)
- Special case: single element queue

3. Common Operations:

- Always check for null/empty before operations
- Update pointers carefully
- Free memory (in languages that need it)
- Handle edge cases (empty, single element)